

SISTEMAS DIGITALES

Linux Processes Tutorial

Every command or program running under Linux is called a *process*. Your applications and even your shell itself are processes. The Linux kernel manages the processes on the system, usually without distinguishing among them. Linux is a multi-tasking system –it allows you to continue to work in the foreground while running one or more jobs in the background. It also runs the processes of many users simultaneously.

1. Viewing Your Processes

The command `ps` will show you the status of your processes.

```
ps
  PID TTY          TIME CMD
2488 pts/1 00:00:00 bash
3870 pts/1 00:00:00 ps
```

`ps` displays the process ID, under `PID`; the control terminal (if any), under `TTY`; the cpu time used by the process so far (including both user and system time), under `TIME`; and finally, an indication of the command that is running under `CMD`.

You can also view the state of the process, which is indicated by a sequence of letters. The man pages for `ps` explain what the letters mean if you want to know. For most purposes, you won't really need to know what the letters mean.

In Linux there is a related command, `pstree`.

2. Working with Processes

In the accompanying archive, there are two compressed directories; one of them is called `01-processes`. Once the archive is uncompressed change to that directory. Inside it there is a `Makefile`. Use `make` to compile the source code in that directory.

The first program you should execute is called `system`; it invokes a shell to execute `ps -ax`; note that once you execute it, the output lists any running processes in the host machine, including both `system` and `ps -ax`. Since the output of the program is quite long, it could be necessary to use `Shift-PgUp` and `Shift-PgDown` to visualise it. We will not use this approach to code our SDL processes since it is inefficient...

The next program to test is called `execl`; it uses the `execl` system call to replace the current process image with the `ps -ax` image. Note that now there is no trace of `execl` in the output of

`ps -ax.`

The third program is called `process`; it uses some system calls to show how to obtain some process attributes, namely, the process Id, the process priority, the Id of the process parent, the environment.

The next program to try is called `fork`; it duplicates its process image using the `fork` system call. Once this system call is executed, you will have two instances of the same process; the child process will execute the `for` loop 5 times while the parent process will execute it 3 times. This means that the parent process will finish first, and the output will be garbled with the command prompt.

To solve this slight aesthetic inconvenience, we can use the `wait` system call in the code for the parent process. This call forces the parent to wait for the child. The `WIFEXITED` macro is used to determine the child's exit status while the `WEXITSTATUS` macro is used to determine the child's exit code.

Linux processes are too heavy for our purposes; scheduling them implies that in a context switch, several items need to be preserved and restored... On the other hand, threads are better suited for what we are looking for; as a matter of fact, they are also called lightweight processes since they have a minimum set of attributes.